Department of Computer Science and Engineering

Indian Institute of Technology, Hyderabad

A Secure Multi-Tenant Identity, Authorization and Enterprise Access Platform

Phase 1: Background, Problem Understanding and Related Work

Submitted by: Krishna Teja (CS23BTECH11028)
Jaya Prakash (CS23BTECH11015)

Course: Design and Analysis of Systems / Project Work

Course Coordinator Mr Praveen Tammana

Project Mentor Mr koundinya

Submission Date: September 2026

Contents

1 Background and Motivation **2**

2 Problem Statement **2**
2.1 Formal Problem Statement 2
2.2 Scope 3
2.3 Key Challenges 3
2.4 Assumptions 4
2.5 Expected Outcomes (Aligned to the Milestones) 4

3 Existing Project Landscape **5**
3.1 Commercial and Open Source Identity Providers 5
3.2 Authorization Systems 5
3.3 Provisoning and Directory Standards 6
3.4 Privacy and Governance Frameworks 6
3.5 Emerging Work on Agent Identity 6
3.6 Gaps We Observe 6

4 Related Work and Reference Material **7**
4.1 OpenID Connect Core 1.0 7
4.2 OAuth 2.0 Security Best Current Practice (RFC 9700) 7
4.3 Zanzibar: Google's Consistent, Global Authorization System 7
4.4 SCIM Protocol (RFC 7644) 8
4.5 NIST Privacy Framework 8
4.6 Comparison Summary 8

5 Justification for the Proposed Project **9**

6 Proposed High Level Approach **10**
6.1 Component View 10
6.2 Data Model Sketch 10
6.3 Roadmap for the Next Phases 11

7 Conclusion **11**

References **12**

1 Background and Motivation

Modern software as a service (SaaS) products almost never live in isolation. A learning platform today has to serve individual students who sign up from a phone, instructors who publish content from a laptop, administrators of the platform itself, and increasingly the employees of large enterprises who are enrolled into the platform by their HR or L&D team. Each of these groups arrives through a different door, expects a different level of control and trusts the system in a different way. A learner just wants to sign in with Google and start a course. An enterprise on the other hand wants their employees to use the company single sign on, wants automatic provisoning when a new hire joins, and wants deprovisoning the moment somebody leaves.

The temptation, especially in early stage products, is to bake identity logic directly into the main application. This works for a short while but starts to break as soon as the product grows. Every new microservice ends up writing its own permission checks, every new tenant asks for a slightly different sign in flow, and every audit or compliance question turns into a manual investigation across many databases. The classical response to this is to separate identity and access management (IAM) into its own platform, so that the rest of the system can simply trust the tokens and decisions it emits.

Our project focuses on exactly this seperation. We are asked to design and build a standalone identity and access platform for a mobile and web learning SaaS, which supports individual learners, instructors, administrators, enterprise employees and organization administrators, and which cleanly splits authentication (who is this), user profile (what do we know about them), tenant membership (which organizations do they belong to and in what role) and authorization (are they allowed to do this specific thing right now).

A second motivation that came up strongly during our mentor discussion is the rise of *agentic access*. Users are beginning to delegate real actions to AI agents. If a learner tells an agent to *buy this course bundle for me* or *download my certificates and email them to my manager*, the platform must be able to recognise that the caller is an agent acting on behalf of the user, verify that the user actually delegated this authority, and constrain what the agent can do. This is not fully covered by classical human oriented OAuth flows and it pushes us to think about identity not just for people but also for programs which have their own credentials, their own scopes and their own audit trail.

Together, these two forces, multi tenancy and agentic access, motivate a fresh look at how a small team can build an identity and access layer that is realistic, secure and extensible, without reinventing everything from scratch. The rest of this report explains the problem in more detail, surveys the existing landscape and the reference material we have been pointed to, and sets out our roadmap for the next phases.

2 Problem Statement

2.1 Formal Problem Statement

Design and implement a standalone identity and access platform that clearly separates the concerns of authentication, user profile management, tenant membership and authorization for a multi tenant

learning SaaS. The platform must support consumer style social sign in, enterprise style federated single sign on, device and session lifecycle management, fine grained authorization that can express role based, attribute based and relationship based rules, tenant provisoning and suspension, consent management and full auditability. It must do all of this without forcing every downstream service in the SaaS to re implement identity logic on its own.

2.2 Scope

The scope covered in the project is intentionally broad, because identity is a horizontal concern, but the boundaries are also clearly defined:

- **In scope:** user registration and sign in for consumers, at least one enterprise style OIDC or SAML federation flow (real or a standards faithful simulator), issuance and rotation of access and refresh tokens, session and device management, remote revocation, tenant provisoning with an administrator bootstrap, role based access control together with attribute based and relationship based rules, consent capture and change, structured audit logging, and adversarial test suites for cross tenant and privilege escalation attacks.
- **Additionally in scope (from mentor feedback):** identity for AI agents that act on behalf of a user, including a way to bind an agent to its principal, restrict its scopes, and revoke it independently.
- **Out of scope for phase 1:** building the actual learning content service, payments, video streaming or grading logic. These are treated as downstream consumers that read from our platform.

2.3 Key Challenges

Building this platform is not a matter of stitching together a login page and a permissions table. The real difficulty comes from several deep problems that interact with each other:

1. **Separation of concerns without loss of context.** Splitting authentication, profile, membership and authorization into different services keeps the design clean, but a policy decision often needs data from all of them at once. Passing this context around efficiently and safely is non trivial.
2. **Tenant isolation.** Two organizations must never be able to read or influence each other's data, even through indirect paths like shared caches, misconfigured tokens or predictable identifiers.
3. **Session and device lifecycle.** Access tokens must be short lived, refresh tokens must rotate, a lost phone must be revocable from another device, and a replayed token must be rejected. All of these have to behave according to a written policy, not just as accidents of implementation.
4. **Fine grained authorization at scale.** Real access rules are rarely just *admin can do everything*. They look more like *an instructor can grade a submission only if the submission belongs to a course they teach in the same organization, and only during the grading window*. Expressing and evaluating this consistently is hard.
5. **Enterprise onboarding.** Provisoning a new organization idempotently, bootstrapping its first administrator, configuring its domain and SSO, and being able to suspend or reactivate it safely, is a workflow with many failure modes.

6. **Privacy and consent.** Users must be able to see and change what they have agreed to, and downstream services should only ever see the fields they are actually allowed to see, not the whole profile.
7. **Agent access.** Recognising that a caller is an AI agent working for a user, and applying the right restricted policy, is a relatively new concern with no single standard yet.
8. **Adversarial validation.** It is not enough for the platform to work on the happy path. We must be able to demonstrate, through automated tests, that cross tenant reads, privilege escalations, revoked sessions and malformed federation claims are rejected.

2.4 Assumptions

To keep the phase 1 understanding focused and honest, we make the following assumptions:

- All identities and organizations used during development and evaluation are synthetic. No real user data is involved.
- External identity providers such as Google, Microsoft Entra ID or Okta may be mocked when actual credentials are not available, but the interfaces and security semantics we implement remain faithful to the real specifications.
- Downstream services (course delivery, grading, certificates) are assumed to trust tokens and decisions issued by our platform, and will not maintain their own parallel user tables.
- The threat model includes external attackers, malicious tenants, curious tenants, revoked users and mis behaving agents. It does not include a fully compromised host operating system.

2.5 Expected Outcomes (Aligned to the Milestones)

The problem statement is tied to seven milestones from M1 to M7. In our understanding, they translate into the following outcomes that must be visible at the end of the project:

- **M1 – Identity and tenant model.** Users, authentication identities, application profiles, organizations, memberships and roles are stored as separate but linked entities, and can be inspected through documented APIs.
- **M2 – Working sign in and federation.** At least two authentication mechanisms work end to end on both mobile and web clients, and at least one of them is an enterprise style OIDC or SAML federation.
- **M3 – Secure session and device lifecycle.** Access token expiry, refresh token rotation, logout, remote device revocation, account reset and replayed token rejection all follow a written policy.
- **M4 – Fine grained authorization.** A declarative policy engine evaluates role, tenant, resource and contextual attributes for at least 40 representative access scenarios.
- **M5 – Enterprise tenant provisoning.** A new organization can be provisioned idempotently, with administrator bootstrap, domain and SSO configuration, quotas and safe suspend or reactivate.

- **M6 – Consent and auditability.** Consent changes, privileged access and security sensitive operations produce queryable audit records, and downstream views expose only permitted profile fields.
- **M7 – Security validation.** Automated adversarial tests prove that cross tenant access, privilege escalation, revoked sessions and malformed federation claims are denied.

The project is considered done when an evaluator can provision two organizations, sign users in through different mechanisms, exercise multi device sessions and enterprise access, change authorization policies, and see through automated tests that tenant and privilege boundaries hold.

3 Existing Project Landscape

Identity is one of the most mature areas of enterprise software, so it is important that we look honestly at what already exists before proposing something of our own. In this section we group the landscape into a few families and then discuss the specific gaps that motivate our project.

3.1 Commercial and Open Source Identity Providers

Products such as Okta, Microsoft Entra ID (formerly Azure AD), Auth0, Ping Identity, Google Cloud Identity and AWS Cognito dominate the enterprise identity market. On the open source side, Keycloak, Ory (Kratos, Hydra, Keto), Zitadel, Authentik, Gluu and FusionAuth are widely used. All of these implement the core protocols we care about, mainly OpenID Connect and OAuth 2.0, and most of them offer SAML for older enterprise integrations. Many also implement SCIM for provisoning.

These systems are excellent, but they are also very large. A student team cannot realistically compete with them on features. What we can do, and what the project actually asks of us, is to build a smaller platform that reflects the same architectural decisions, so we understand why these systems look the way they do. In particular, most commercial IdPs still treat authorization as a fairly coarse feature (roles and groups), and they push fine grained decisions back to the applications. This is one of the gaps we want to address.

3.2 Authorization Systems

The authorization landscape is younger and more fragmented. Traditional role based access control (RBAC) is built into almost every framework. Attribute based access control (ABAC) is expressed in standards such as XACML, which is powerful but famously hard to use. More recently, relationship based access control (ReBAC), inspired by Google's Zanzibar system, has become popular through open source projects like SpiceDB (AuthZed), OpenFGA (Auth0/Okta), Ory Keto and Warrant. Policy as code engines like Open Policy Agent (OPA) and its Rego language are widely used to express ABAC style rules.

The interesting observation here is that no single model wins. Real systems mix all three: roles for the common case, attributes for context sensitive rules like time or IP, and relationships for things like *the user who created this document can share it*. A modern identity and access platform therefore needs a policy layer that can express and combine all three, which is what milestone M4 is really asking for.

3.3 Provisoning and Directory Standards

SCIM (System for Cross domain Identity Management), defined in RFC 7643 and RFC 7644, is the de facto standard for pushing user and group information from a source of truth (like an HR system or an enterprise IdP) into a SaaS application. Almost every serious enterprise integration today expects SCIM. Older systems still use LDAP or custom feeds, but SCIM is where new work happens. Our platform will need at least a SCIM style provisoning interface to be credible as an enterprise ready system.

3.4 Privacy and Governance Frameworks

Regulations such as the GDPR in Europe and the DPDP Act in India, together with frameworks like the NIST Privacy Framework, shape how identity systems must handle personal data. These are not protocols but they strongly influence design: they demand purpose limitation, data minimisation, consent management, and the ability to answer questions like *what data do you have about me, why do you have it, and who accessed it*. The NIST Privacy Framework in particular gives a structured way to think about identify, govern, control, communicate and protect functions around personal data, which maps very naturally onto milestone M6.

3.5 Emerging Work on Agent Identity

The idea of AI agents acting on behalf of users is very new and there is not yet a widely accepted standard. Early efforts include Anthropic's Model Context Protocol which touches on how agents connect to tools, OAuth style extensions that discuss *on behalf of* tokens, and the general OAuth 2.0 Token Exchange specification (RFC 8693) which was originally meant for service to service delegation but is being repurposed for agent scenarios. Since our mentor explicitly asked us to consider this, we plan to treat agents as first class principals in our data model, give them their own credentials, and require an explicit delegation record that ties an agent to a user and to a set of scopes.

3.6 Gaps We Observe

Looking across this landscape, the gaps that we can genuinely contribute to as a student project are the following:

- Many small and mid sized systems still couple authentication and authorization, which makes tenant isolation and policy changes risky. A clear seperation is worth demonstrating end to end.
- Very few teaching or reference implementations combine RBAC, ABAC and ReBAC in one place with a real test suite that shows they cooperate correctly.
- Agent identity is barely covered by textbook examples, even though it is becoming a common real world requirement.
- Auditability and consent are often afterthoughts in student projects, and we want to treat them as first class from day one.

4 Related Work and Reference Material

The project brief points us to five specific references. In this section we summarise each one and explain how it maps onto the design we are going to build. These references are also the vocabulary boundary for our design work, so we will stay within their concepts.

4.1 OpenID Connect Core 1.0

OpenID Connect (OIDC) is an identity layer built on top of OAuth 2.0. Where OAuth 2.0 only tells a client that it has been granted access to some resource, OIDC additionally tells the client *who* the user is, through a signed ID token. The specification defines the authorization code flow, the implicit flow (now discouraged), the hybrid flow, the structure of the ID token (a JSON Web Token with standard claims like `sub`, `iss`, `aud`, `iat`, `exp` and `nonce`), the UserInfo endpoint, and discovery through the well known configuration document.

For our platform, OIDC is the primary authentication protocol. Consumer sign in (*sign in with Google, sign in with Microsoft*) is OIDC. Enterprise SSO in modern IdPs is usually OIDC as well, with SAML kept for older customers. The ID token is what we will hand to the mobile and web clients so they can prove who the user is, and the access token is what downstream services will validate. The `nonce` and `state` parameters, the requirement to validate the signature, issuer and audience, and the recommendation to use the authorization code flow with PKCE for public clients (including mobile) are all directly relevant to milestones M2 and M3.

4.2 OAuth 2.0 Security Best Current Practice (RFC 9700)

RFC 9700 is the current best practice document for securing OAuth 2.0 deployments. It updates and consolidates a long list of earlier advice. Its main recommendations that we plan to follow include: always use the authorization code flow with PKCE, even for confidential clients; do not use the implicit flow or the resource owner password credentials flow; use exact redirect URI matching; bind refresh tokens to the client and rotate them on every use; detect refresh token reuse and revoke the whole chain when reuse is seen; use sender constrained tokens (DPoP or mTLS) where possible; and treat access tokens as short lived credentials.

This document is essentially the checklist for milestone M3. The behaviours it describes (short lived access tokens, rotating refresh tokens, detecting replayed tokens, revoking devices, and safely logging users out) form the backbone of our session and device lifecycle policy. Any deviation from RFC 9700 in our design will be a conscious and documented decision.

4.3 Zanzibar: Google's Consistent, Global Authorization System

Zanzibar is the paper that describes how Google runs authorization at planet scale for products like Drive, YouTube, Calendar and Photos. Its core idea is to model permissions as *relations* between objects and users, stored as tuples of the form `(object, relation, user)`. A permission check becomes a graph reachability question over these tuples. The paper explains how they achieve external consistency using *zookies* (opaque tokens that pin a check to a specific version of the data), how they scale reads

with caching and Leopard indexes, and how they support namespaces so different products can define their own relation schemas.

Zanzibar is the intellectual root of the ReBAC systems mentioned earlier. For our platform, it directly shapes the authorization engine we plan to build for milestone M4. We do not need Google scale, but the tuple model, the concept of namespaces per resource type, and the discipline of separating *who has what relation to what* from *who can do what* are ideas we intend to adopt. Combined with RBAC (for common roles) and ABAC (for time, IP, device posture) we get a policy engine that can genuinely express the kind of rules a learning SaaS needs.

4.4 SCIM Protocol (RFC 7644)

SCIM defines a REST API for managing users and groups across systems. RFC 7644 covers the protocol (endpoints, filters, bulk operations, patch semantics, error responses), while RFC 7643 covers the schema (User, Group, EnterpriseUser). An enterprise IdP typically pushes lifecycle events to a SaaS through SCIM: create a user when a new employee joins, update their attributes when they change teams, and deactivate them when they leave.

For our project, SCIM is directly relevant to milestone M5. When an organization is provisioned on our platform and connects its enterprise IdP, we want new employees to appear automatically and departed employees to be deactivated automatically. Implementing at least the core SCIM endpoints (`/Users`, `/Groups`, with GET, POST, PATCH, DELETE) gives us a realistic enterprise ready surface without inventing a proprietary API.

4.5 NIST Privacy Framework

The NIST Privacy Framework is a voluntary framework, similar in spirit to the NIST Cybersecurity Framework, that helps organizations manage privacy risk. It is organised around five functions: Identify, Govern, Control, Communicate and Protect. Under each function it defines categories and subcategories that describe outcomes an organization should be able to demonstrate, such as maintaining an inventory of data processing, defining data processing policies, giving individuals mechanisms to manage their data, and monitoring for privacy events.

For our platform, the framework is the guiding structure for milestone M6. Concretely, it tells us that we need: a clear inventory of what personal data we store on each user, purpose specific consent records that can be changed by the user, controls that restrict which downstream service can see which fields, communication to the user about how their data is used, and audit logs that let us investigate privacy events. We will not claim full NIST compliance, but we will map our consent and audit features onto its vocabulary so evaluators can see the connection.

4.6 Comparison Summary

Table 1 summarises how each reference maps to the milestones of the project.

Table 1: Mapping of references to project milestones

Reference	Primary Concern	Milestones it informs
OpenID Connect Core 1.0	Authentication and identity assertions	M2 (sign in and federation), M3 (session lifecycle)
OAuth 2.0 BCP (RFC 9700)	Secure token handling	M3 (sessions), M7 (security validation)
Zanzibar	Fine grained authorization at scale	M4 (policy engine), M7 (adversarial tests)
SCIM (RFC 7644)	Enterprise provisoning	M1 (identity model), M5 (tenant provisoning)
NIST Privacy Framework	Privacy governance and audit	M6 (consent and audit)

5 Justification for the Proposed Project

Given that mature commercial products already exist, it is fair to ask why we should build our own platform rather than simply integrating one of them. Our justification is not that we can replace Okta or Keycloak, but that a purpose built, well scoped platform lets us demonstrate several things that off the shelf products hide behind their user interface.

First, the project forces us to *separate* authentication, profile, membership and authorization into distinct services with well defined APIs. Doing this by hand teaches us where the seams should be, which is exactly the kind of understanding a systems engineer needs. Using a black box IdP would give us working sign in but no insight into these seams.

Second, the requirement to combine RBAC, ABAC and ReBAC in one policy engine is unusual. Most tutorials show only one of them. By building a small engine that can express all three, and by writing at least 40 representative access scenarios as tests, we can show that the boundaries between the three models are not as sharp as textbooks suggest.

Third, the mentor's requirement to treat AI agents as first class principals is genuinely new. There is not yet a settled industry answer, and there is real value in building a small, honest prototype that models an agent as a distinct principal, bound to a human user through an explicit delegation record, with its own token, its own scopes and its own audit trail. This is a contribution that even production IdPs are still figuring out.

Fourth, the emphasis on adversarial testing (milestone M7) is unusual in student projects. Building automated tests that try to break tenant isolation, escalate privileges, replay revoked tokens and inject malformed federation claims forces us to think like an attacker and not just like a developer. This is exactly the discipline the OAuth 2.0 Security BCP is trying to encourage.

Finally, the emphasis on consent and audit through the NIST Privacy Framework connects the project to the growing regulatory reality (GDPR, DPDP Act). Even a small demonstration that shows queryable audit logs, downstream views limited to permitted fields, and user controlled consent, is a valuable exercise in privacy engineering.

For all these reasons, we believe the project is well justified. It does not compete with commercial products; it complements them by making the underlying design choices visible, testable and extendable

in directions (agents, ReBAC, adversarial tests, privacy) where the industry itself is still catching up.

6 Proposed High Level Approach

Although this is a phase 1 report, our understanding is much clearer when we can point to the rough shape of the system we intend to build. This section is intentionally at a high level; detailed design comes in phase 2.

6.1 Component View

We plan to organise the platform into the following logical components, each with its own responsibility and its own API:

- **Authentication Service.** Owns login, federation, MFA and password reset. Speaks OIDC to clients and to upstream IdPs. Issues ID tokens and access tokens.
- **User and Profile Service.** Owns the canonical user record and application profile fields. Enforces field level visibility for downstream services.
- **Tenant and Membership Service.** Owns organizations, memberships, roles and enterprise configuration (domain, SSO, quotas). Handles provisoning and suspension.
- **Authorization Service.** A Zanzibar inspired policy engine that stores relation tuples and evaluates decisions combining RBAC, ABAC and ReBAC. Exposes a `Check`, `Expand` and `Write` API.
- **Session and Device Service.** Manages access tokens, rotating refresh tokens, device records and revocation. Enforces the policy from RFC 9700.
- **SCIM Provisoning Endpoint.** Exposes SCIM 2.0 style endpoints for enterprise IdPs to push user and group changes.
- **Consent and Audit Service.** Records purpose specific consents, tracks changes, and stores structured audit events for privileged and security sensitive operations.
- **Agent Delegation Service.** Registers agents, binds them to a user principal, issues restricted tokens for them, and lets users revoke them.

6.2 Data Model Sketch

At the entity level, the model separates:

- **User** – the person, uniquely identified.
- **Authentication Identity** – one per login method (email plus password, Google, enterprise SSO), linked to a user.
- **Profile** – application specific attributes, linked to a user, with field level visibility rules.
- **Organization** – a tenant, with its own configuration.

- **Membership** – the link between a user and an organization, carrying one or more roles.
- **Role** – a named bundle of permissions inside an organization.
- **Agent** – a program principal linked to a user through a delegation record.
- **Relation Tuple** – the Zanzibar style `(object, relation, subject)` record used by the authorization engine.
- **Session and Device** – token state for lifecycle enforcement.
- **Consent Record and Audit Event** – privacy and governance state.

This seperation directly satisfies milestone M1 and is what makes the rest of the system possible.

6.3 Roadmap for the Next Phases

Table 2: Tentative roadmap for the remaining phases

Phase	Focus	Expected Deliverables
Phase 2	Detailed design and data model	Component diagrams, API specifications, ER diagrams, policy schema, threat model
Phase 3	Core implementation	Authentication service with OIDC, user and tenant services, basic RBAC, mobile and web sample clients
Phase 4	Advanced authorization and enterprise features	Zanzibar style ReBAC engine, ABAC context, SCIM endpoint, enterprise SSO simulator, agent delegation
Phase 5	Security validation, audit and documentation	40+ access scenario tests, adversarial test suite, consent and audit UI, final report and demo

7 Conclusion

This report has documented our understanding of the assigned project: a secure, multi tenant identity, authorization and enterprise access platform for a learning SaaS, extended to also cover AI agents that act on behalf of users. We explained the background and why identity deserves to be a standalone platform rather than something each service reinvents. We defined the problem precisely, listed the scope, the challenges, the assumptions and the milestone driven outcomes. We surveyed the existing landscape of commercial and open source identity providers, authorization systems, provisoning standards, privacy frameworks and the emerging space of agent identity. We summarised each of the five reference sources and mapped them explicitly onto our milestones. Finally, we justified why a purpose built platform, even at student scale, is worth building, and we sketched the components and roadmap we intend to follow.

We have discussed this understanding with our mentor and have incorporated the specific feedback about agent identity into both our scope and our data model. In the next phase we will move from understanding to detailed design, converting the components sketched here into concrete APIs, schemas and a threat model that the rest of the implementation can be built against.

References

1. Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., Mortimore, C. *OpenID Connect Core 1.0.* OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html

2. Lodderstedt, T., Bradley, J., Labunets, A., Fett, D. *OAuth 2.0 Security Best Current Practice (RFC 9700).* IETF. https://www.rfc-editor.org/rfc/rfc9700.html

3. Pang, R., Caceres, R., Burrows, M., et al. *Zanzibar: Google's Consistent, Global Authorization System.* USENIX ATC 2019. https://www.usenix.org/conference/atc19/presentation/pang

4. Hunt, P., Grizzle, K., Ansari, M., Wahlstroem, E., Mortimore, C. *System for Cross domain Identity Management: Protocol (RFC 7644).* IETF. https://www.rfc-editor.org/rfc/rfc7644.html

5. National Institute of Standards and Technology. *NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management.* https://www.nist.gov/privacy-framework